# LAPORAN PRAKTIKUM
# PRAKTIKUM PENGUJIAN PERANGKAT LUNAK
# PERTEMUAN 3
# PARAMETERIZED TEST PADA JUNIT 5

**Disusun oleh:**


| | |
|---|---|
| **Nama** | : Januarsyah Akbar |
| **NIM** | : 24/535846/SV/24314 |
| **Kelas** | : PL4B2 |
| **Dosen Pengampu** | : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D. |

**PROGRAM STUDI D-IV**
**TEKNOLOGI REKAYASA PERANGKAT LUNAK**
**DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA**
**SEKOLAH VOKASI**
**UNIVERSITAS GADJAH MADA**
**YOGYAKARTA**


**March 3, 2026**

# Daftar Isi

**Daftar Isi** . . . . . **2**

**Tujuan Praktikum** . . . . . **3**

**Hasil dan Pembahasan** . . . . . **4**
2.1 Percobaan Latihan Praktikum . . . . . 4
2.1.1 Pengujian Method `checkEven` dengan `@ValueSource` . . . . . 4
2.1.2 Pengujian Method `tambah` dengan `@CsvSource` . . . . . 5
2.1.3 Pengujian Method `kurang` dengan `@MethodSource` . . . . . 6
2.2 Tugas dan Analisis Praktikum . . . . . 7
2.2.1 Validasi Nominal Cash dengan Single-value Parameter . . . . . 7
2.2.2 Pengujian Transaksi Withdraw untuk Data Valid dan Invalid . 8
2.2.3 Implementasi Owner sebagai Object dan Pengujian MethodSource 10

**Resource** . . . . . **12**

**Kesimpulan** . . . . . **13**

# Tujuan Praktikum

1. Mahasiswa mampu memahami tipe-tipe parameter pada test method di JUnit 5.
2. Mahasiswa mampu mengimplementasikan *parameterized test* menggunakan `@ValueSource`, `@CsvSource`, dan `@MethodSource`.
3. Mahasiswa mampu menerapkan *parameterized test* pada studi kasus *Calculator* dan *Wallet*.

# Hasil dan Pembahasan

## 2.1 Percobaan Latihan Praktikum

### 2.1.1 Pengujian Method `checkEven` dengan `@ValueSource`

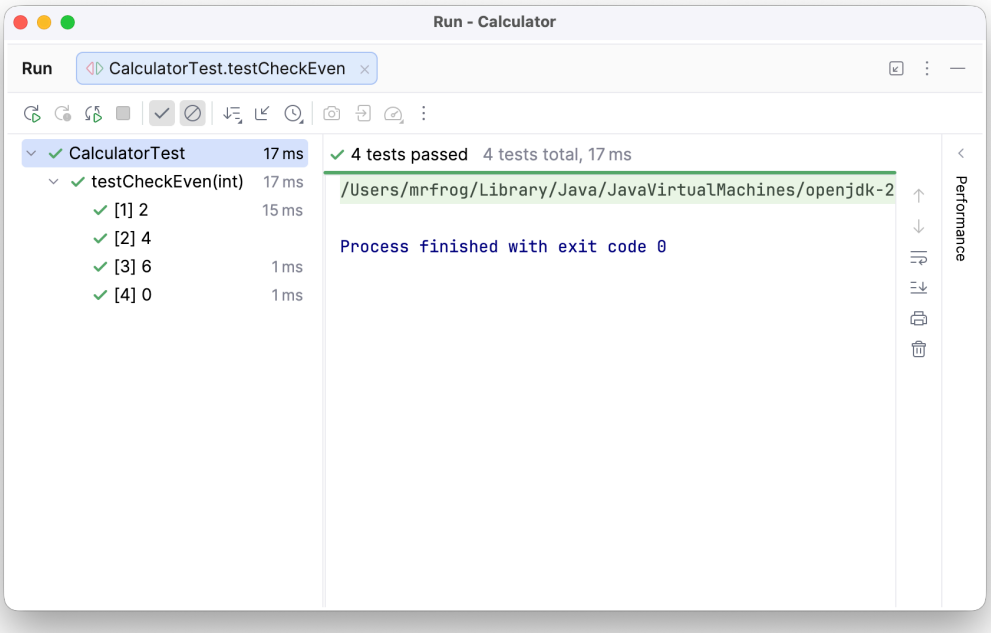
Pada studi kasus *Calculator*, method `checkEven(int number)` digunakan untuk memeriksa apakah suatu bilangan bernilai genap. Pengujian dilakukan dengan *parameterized test* menggunakan beberapa input genap.

```
public boolean checkEven(int number) {
    return number % 2 == 0;
}
```

Listing .1: Implementasi method checkEven pada class Calculator

```
@ParameterizedTest
@ValueSource(ints = {2, 4, 6, 0})
void testCheckEven(int inits) {
    Calculator cal = new Calculator(inits, 0);
    assertTrue(cal.checkEven(inits));
}
```

Listing .2: Parameterized test ValueSource untuk checkEven



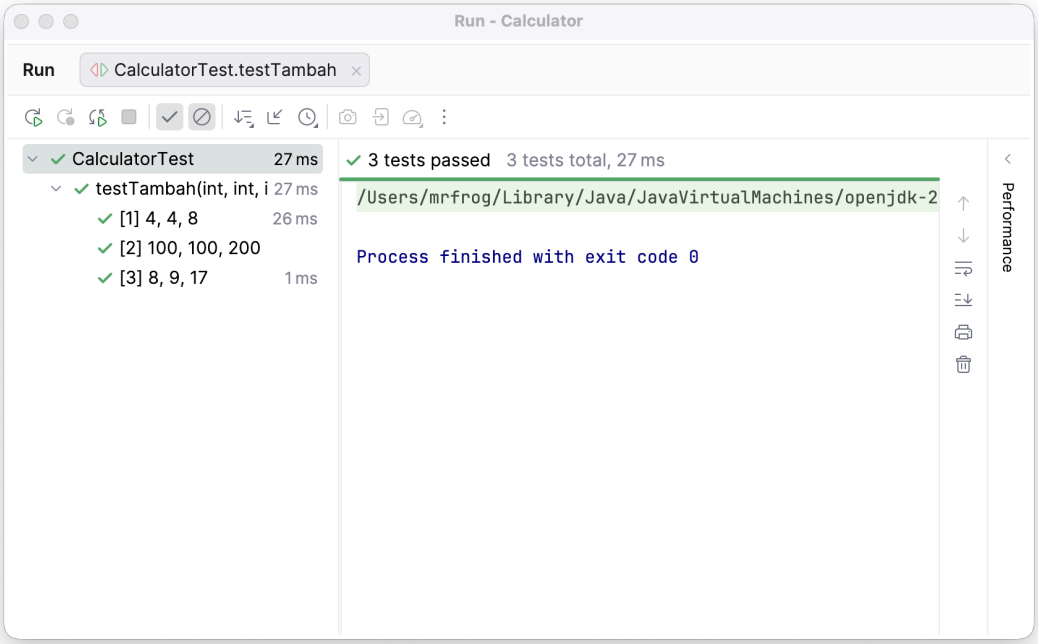

Gambar 1: Hasil eksekusi parameterized test menggunakan ValueSource

### 2.1.2 Pengujian Method `tambah` dengan `@CsvSource`

Pengujian ini menggunakan tiga parameter sekaligus dalam format CSV yaitu nilai `a`, `b`, dan nilai *expected*. Setiap baris data CSV akan dieksekusi sebagai satu skenario pengujian.

```
@ParameterizedTest
@CsvSource({"4,4,8", "100,100,200", "8,9,17"})
void testTambah(int a, int b, int expected) {
    Calculator cal = new Calculator(a,b);
    int actual = cal.tambah();
    assertEquals(expected, actual);
}
```

Listing .3: Parameterized test CsvSource untuk method tambah



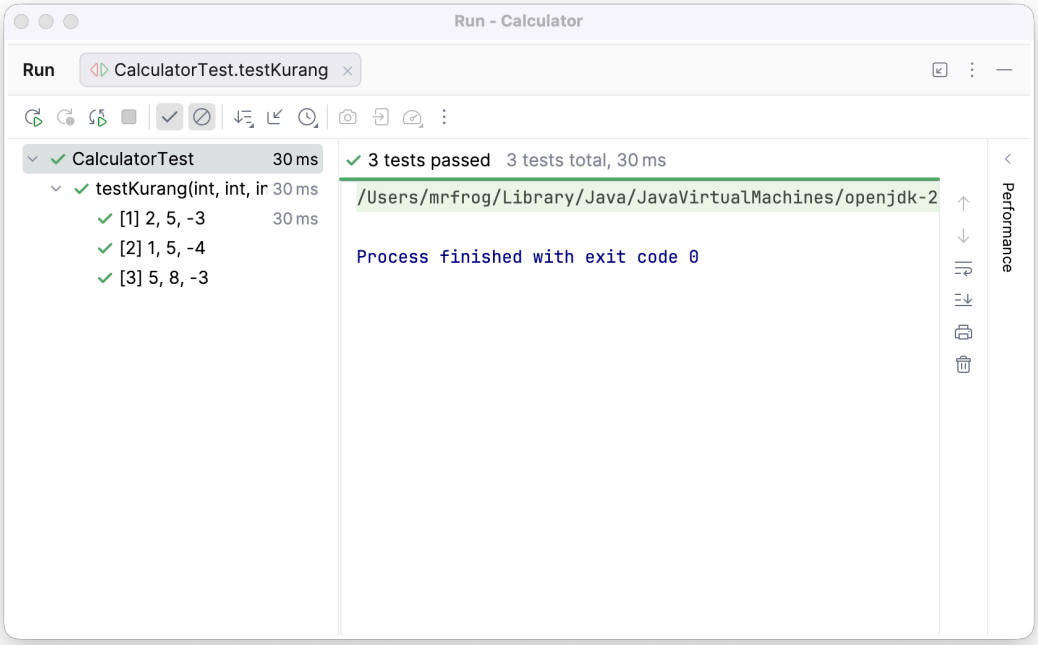

Gambar 2: Hasil eksekusi parameterized test menggunakan CsvSource

### 2.1.3 Pengujian Method `kurang` dengan `@MethodSource`

Untuk `@MethodSource`, data parameter disediakan dari method terpisah bernama `data()`. Method ini mengembalikan *stream* berisi kombinasi argument pengujian.

```
private static Stream<Arguments> data() {
    return Stream.of(
        Arguments.of("2","5","-3"),
        Arguments.of("1","5","-4"),
        Arguments.of("5","8","-3")
    );
}

@ParameterizedTest
@MethodSource("data")
void testKurang(int a, int b, int expected) {
    Calculator cal = new Calculator(a,b);
    int actual = cal.kurang();
    assertEquals(expected, actual);
}
```

Listing .4: Penyedia data dan test MethodSource untuk method kurang




Gambar 3: Hasil eksekusi parameterized test menggunakan MethodSource

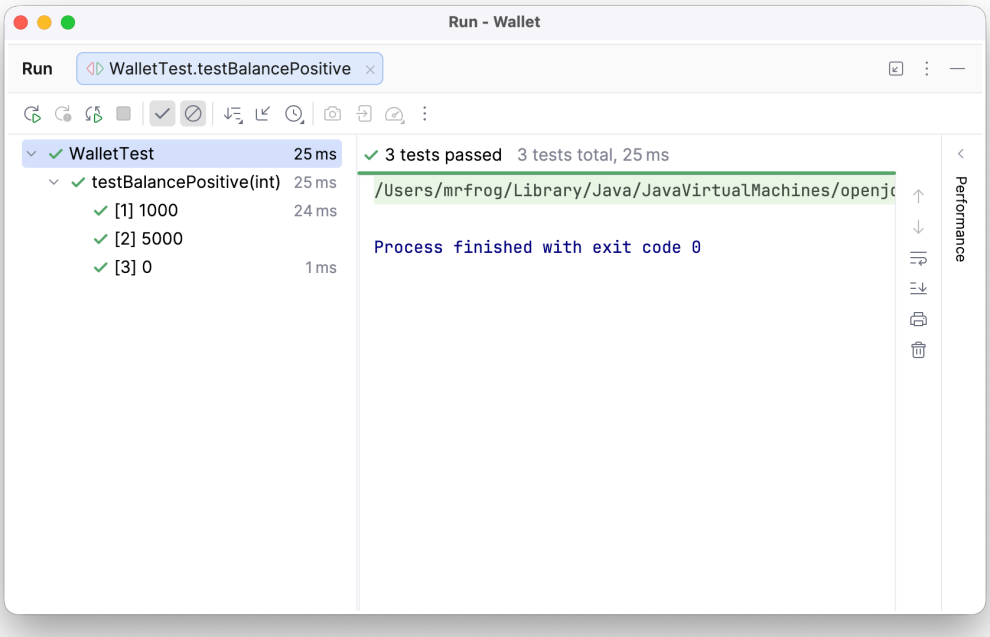
## 2.2 Tugas dan Analisis Praktikum

### 2.2.1 Validasi Nominal Cash dengan Single-value Parameter

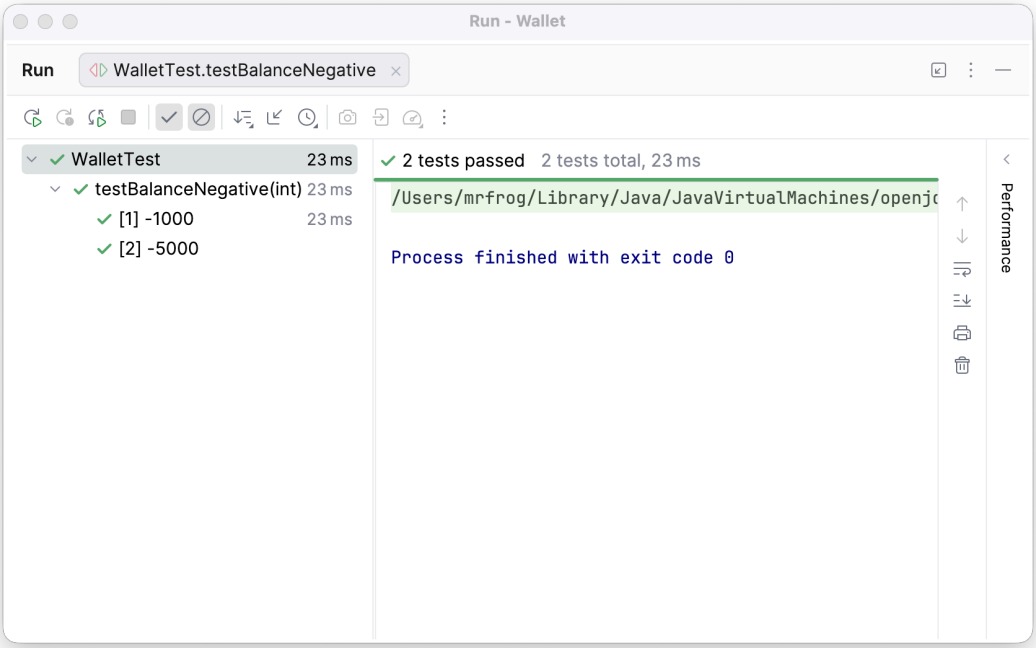
Pada studi kasus *Wallet*, validasi nominal dilakukan melalui method `validBalance(int amount)`. Method mengembalikan `true` untuk nilai tidak negatif, dan `false` untuk nilai negatif. Pengujian dilakukan menggunakan `@ValueSource` untuk data valid dan tidak valid.

```
public boolean validBalance(int amount) {
    if (amount < 0) {
        return false;
    }
    return true;
}

@ParameterizedTest
@ValueSource(ints = {1000, 5000, 0})
void testBalancePositive(int amount) {
    Wallet wallet = new Wallet();
    assertTrue(wallet.validBalance(amount));
}

@ParameterizedTest
@ValueSource(ints = {-1000, -5000})
void testBalanceNegative(int amount) {
    Wallet wallet = new Wallet();
    assertFalse(wallet.validBalance(amount));
}
```

Listing .5: Method validBalance dan pengujiannya dengan ValueSource




Gambar 4: Hasil pengujian nominal valid (balance positive)




Gambar 5: Hasil pengujian nominal tidak valid (balance negative)

### 2.2.2 Pengujian Transaksi Withdraw untuk Data Valid dan Invalid

Method `withdraw(int amount)` memiliki validasi khusus: nilai negatif dan nol akan melempar `IllegalArgumentException`, sedangkan nilai melebihi saldo akan

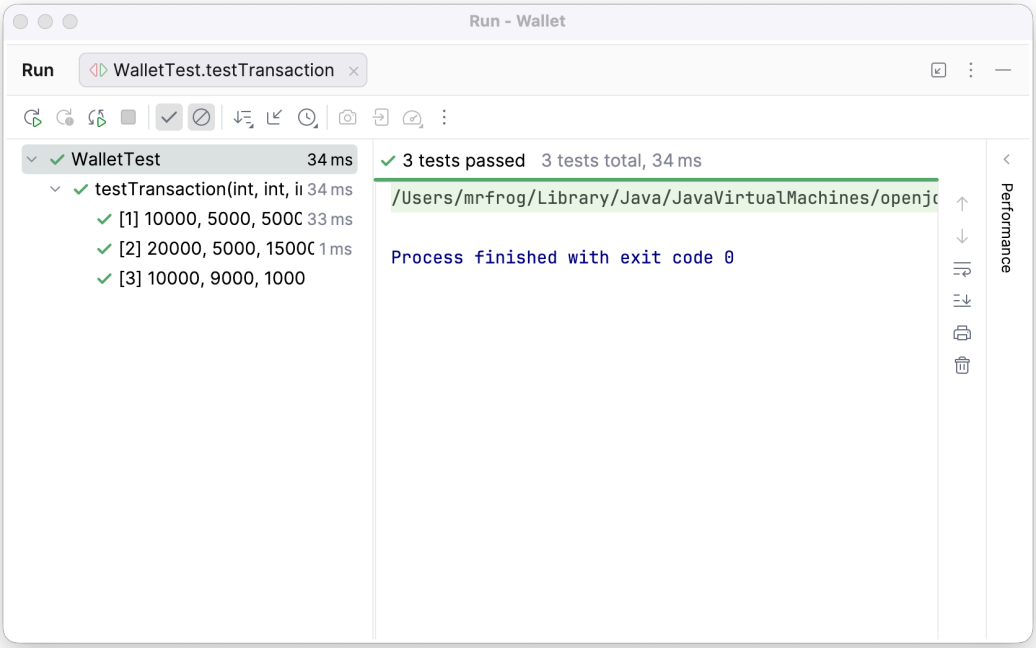
melempar `InsufficientFundsException`. Skenario valid dan invalid diuji secara parameterized.

```java
public boolean withdraw(int amount) {
    if (amount < 0) {
        throw new IllegalArgumentException("Amount cannot be negative");
    }
    if (amount > totalMoney) {
        throw new InsufficientFundsException("Amount cannot be greater than total money");
    }
    if (amount == 0) {
        throw new IllegalArgumentException("Amount cannot be zero");
    }
    totalMoney -= amount;
    return false;
}
```

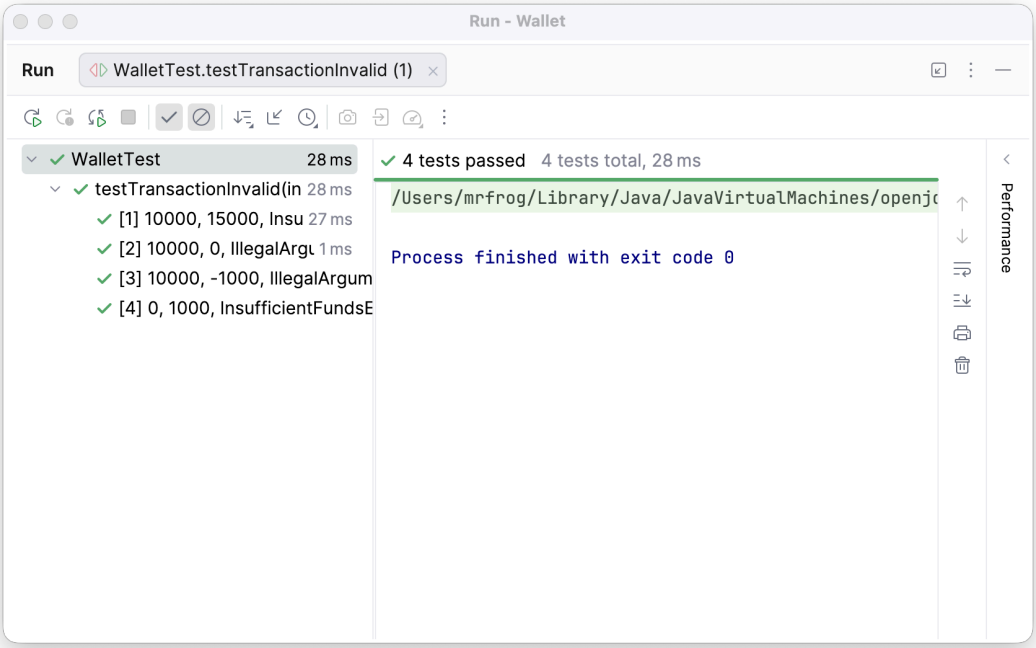
Listing .6: Implementasi withdraw pada class Wallet

```java
@ParameterizedTest
@CsvSource({"10000,5000,5000", "20000,5000,15000", "10000,9000,1000"})
void testTransaction(int deposit, int withdraw, int expected) {
    wallet.deposit(deposit);
    wallet.withdraw(withdraw);
    assertEquals(expected, wallet.getTotalMoney());
}

@ParameterizedTest
@CsvSource({
    "10000,15000,InsufficientFundsException",
    "10000,0,IllegalArgumentException",
    "10000,-1000,IllegalArgumentException",
    "0,1000,InsufficientFundsException"
})
void testTransactionInvalid(int deposit, int withdraw, String exceptionType) {
    if (deposit > 0) {
        wallet.deposit(deposit);
    }

    if (exceptionType.equals("InsufficientFundsException")) {
        assertThrows(InsufficientFundsException.class, () -> wallet.withdraw(withdraw));
    } else {
        assertThrows(IllegalArgumentException.class, () -> wallet.withdraw(withdraw));
    }
}
```

Listing .7: Parameterized test transaksi valid dan invalid




Gambar 6: Hasil pengujian transaksi valid




Gambar 7: Hasil pengujian transaksi invalid dengan assertion exception

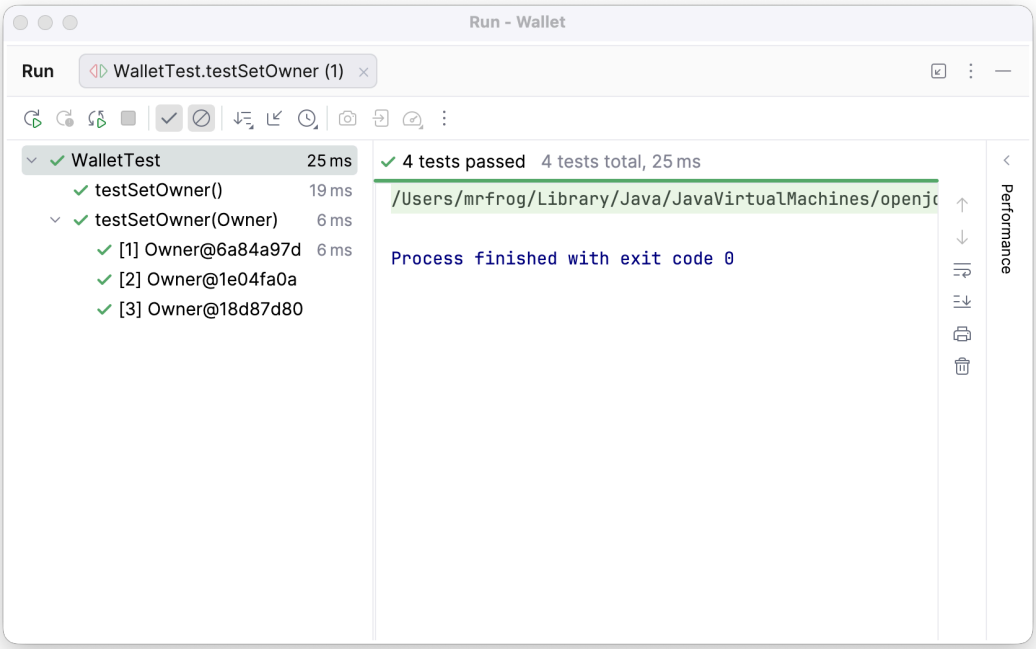
### 2.2.3 Implementasi Owner sebagai Object dan Pengujian MethodSource

Sesuai ketentuan tugas, atribut *owner* tidak lagi berupa *string*, melainkan object dari class `Owner` yang memiliki `id`, `name`, dan `email`. Pengujian `setOwner(Owner`

owner) dilakukan menggunakan `@MethodSource` dengan *argument provider* object.

```java
public class Owner {
    private final String id;
    private final String name;
    private final String email;

    public Owner(String id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }
}

private static Stream<Arguments> ownerProvider() {
    return Stream.of(
        Arguments.of(new Owner("1", "Budi", "budi@mail.com")),
        Arguments.of(new Owner("2", "Andi", "andi@mail.com")),
        Arguments.of(new Owner("3", "Sinta", "sinta@mail.com"))
    );
}

@ParameterizedTest
@MethodSource("ownerProvider")
void testSetOwner(Owner owner) {
    Wallet wallet = new Wallet();
    wallet.setOwner(owner);
    assertEquals(owner, wallet.getOwner());
}
```

Listing .8: Class Owner dan pengujian setOwner dengan MethodSource




Gambar 8: Hasil pengujian setOwner dengan MethodSource

# Resource

Repository source code praktikum ini dapat diakses melalui GitHub berikut:

`https://github.com/januarsyah901/PPPL/tree/822cf2bb606fc8fd108707b2a7ef142aff78af1e/Pertemuan%203`

# Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *parameterized test* pada JUnit 5 mempermudah pengujian banyak skenario dalam satu method uji. Penggunaan `@ValueSource`, `@CsvSource`, dan `@MethodSource` berhasil diterapkan pada studi kasus *Calculator* dan *Wallet*. Implementasi pengujian untuk kasus valid dan invalid, termasuk pengujian exception, membantu memastikan perilaku sistem lebih konsisten. Selain itu, perubahan *owner* menjadi object class `Owner` meningkatkan struktur data dan fleksibilitas pengembangan lanjutan.